1 2 9 0

FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE D
COIMBRA

Departamento de Engenharia Informática

Intelligent Autonomous Agents
2025/2026

3rd Project
Searching for Pac-Man

Fraud denotes a serious lack of ethics and represents unacceptable behavior for a higher education student. Any attempt at fraud will result in the cancellation of the practical component for both the facilitator and the perpetrator, regardless of any additional disciplinary actions that may be taken under current legislation. If non-original material is used, the **sources** must be explicitly cited. If you utilize AI tools in the production of this work (e.g., ChatGPT), you must clearly specify all aspects in which the tool was involved. Note that, during the defense, you must demonstrate a deep understanding of the content generated by the tool, and this knowledge will be subject to evaluation.

1 Introduction

The third project builds on top of the previous ones. In it, a Pac-Man reactive agent is provided and we will focus on improving the ghosts. As you have probably seen, there is a big problem with the current ghost agents, as Blinky, Pinky and Clyde will be stuck in their home base until Pac-Man goes upward of them or eats a Power Pellet. This is a consequence of the current implementation of their behaviour, which selects the action that minimises the distance to their goal location (which is either Pac-Man's location or close to it). As the only option to leave the home base is moving up, and that direction leads the ghosts away from Pac-Man, it will never be chosen.

In this project you are asked to use your knowledge of **search methods** to improve on these limitations, while **maintaining the overall ghost behaviours**. However, for the sake of simplicity, **a few constraints may be relaxed**. For instance, we will allow the ghosts to reverse their direction of motion, avoiding to insert that restriction into the search algorithm. Likewise, we may allow ghosts to remain stationary, which will be particularly useful for when they are running away from Pac-Man.

2 Tasks

The goal of this assignment is for you to improve the ghost's ability to navigate the environment by using search algorithms to compute their paths. The overall behaviours of the ghosts should be preserved, so that they behave as follows:

- Blinky – This ghost moves towards Pac-Man's current position.
- Pinky - This ghost's personality is characterized by ambush attempts, trying to move in a way that intercepts Pac-Man's movement. To achieve this, Pinky moves towards four cells ahead of Pac-Man.
- Inky - This is the most erratic ghost, performing a random walk in the environment.
- Clyde – This ghost is the most paradoxical of all, as it both wants to chase Pac-Man and is simultaneously afraid of him. When more than five cells away from Pac-Man, Clyde moves toward him. Otherwise, it moves randomly.

Furthermore, when any ghost is frightened, it will move to the location furthest away from Pac-Man. Your task is to use your knowledge regarding

search algorithms to improve the ghosts' behaviours, enabling them to avoid getting stuck. Specifically, you should implement the Blinky, Pinky, Inky and Clyde search agents, as well as the frightened search agent. To do so, you must start by analysing the TP3 lines in pacman.py, as shown in the listing below.

```
16 n_experiments = 30
17 search_algorithms = ['bfs', 'dfs', 'greedy', 'a_star']
18
19 for search_algorithm in search_algorithms:
20     success_rate =0
21     scores = []
22     print('---search_algorithm: ', search_algorithm,'---')
23
24     for experiment in range(n_experiments):
25
26         pacman_policy = agents.pacman_reactive_agent
27         ghost_policies = [agents.blinky_search_agent(
               search_algorithm), agents.pinky_search_agent(
               search_algorithm), agents.inky_search_agent(
               search_algorithm), agents.clyde_search_agent(
               search_algorithm)]
28         frightened_ghost_policies = [agents.
               run_away_from_pacman_search(search_algorithm) for _
               in range(4)]
29         game_state = game_engine.main(pacman_policy,
               ghost_policies, frightened_ghost_policies, map_file='
               maps/originalClassic.txt')
30
31
32         for ghost in game_state['ghosts']:
33             print(ghost['name'], 'mean path lengths: ', np.mean(
                   ghost['path_lengths']), 'no path found rate: ',
                   ghost['no_path_found']/game_state['time_step'], '
                   stationary rate: ', ghost['stationary']/
                   game_state['time_step'], 'moving rate: ', ghost['
                   moving']/game_state['time_step'])
34
35         scores.append(game_state['score'])
36
37     print(f"Average score of {n_experiments} experiments: ", np.
           mean(scores), 'success rate: ', success_rate/
           n_experiments)
```

The code provided already runs all search algorithms for 30 experiments. It also prints some of the stored statistics relevant for yout analysis (lines 33 and 37). As you can see, for each ghost, all path lengths computed during each simulation are stored, and so is the number of times that the search

algorithm could not find a path. Also, depending on the goal and start positions, it is possible that the ghost becomes stationary or moving. Those events are also counted.

The search algorithms and ghost agents must be implemented in the corresponding functions in agents.py, i.e.:

- breadth_first_search
- bepth_first_search
- greedy_search
- a_star_search_search
- blinky_search_agent
- pinky_search_agent
- inky_search_agent
- clyde_search_agent
- run_away_from_pacman_search

The listing below shows the blinky_search_agent function. As you can seen, the steps are already laid out for you. Whenever you believe that you need to compute a path, you should start by defining the start and goal positions and the proceed to call the compute_path function. Finally, you should use the move_in_path function to move one step in that path.

```
153 def blinky_search_agent(search_algorithm):
154     def agent(ghost, game_state):
155         #TODO: Implement the search agent for Blinky
156         #1. Define the start and goal positions (as tuples of x,y
                )
157
158         #2. Use the compute_path function to compute the path
                from start to goal
159
160         #3. Use the move_in_path function to move the ghost in
                the first step of the path
161         pass
162     return agent
```

The listing below shows the compute_path function. This function is already fully implemented and, given a start and goal position, a transition graph and a search algorithm computes calls the corresponding search algorithm function to compute the path (lines 99 to 108). The agent object is also passed as a parameter to save data for later analysis (lines 110 to 116).

```python
def compute_path(start, goal, transition_graph, search_algorithm, agent):
    #This function is called by the search agents to compute the path from start to goal. It is already fully implemented
    path = None
    if search_algorithm == 'bfs':
        path = breadth_first_search(start, goal, transition_graph)
    elif search_algorithm == 'dfs':
        path = depth_first_search(start, goal, transition_graph)
    elif search_algorithm == 'greedy':
        path = greedy_search(start, goal, transition_graph)
    elif search_algorithm == 'a_star':
        path = a_star_search(start, goal, transition_graph)
    else:
        raise ValueError("Unknown search algorithm")

    if path is None:
        agent['no_path_found'] += 1
    elif start == goal:
        agent['stationary'] +=1
    else:
        agent['moving'] +=1
        agent['path_lengths'].append(len(path))

    return path
```

As you can see, the compute_path_function calls the intended search algorithm. As an example, the listing below shows the breadth first function.

```python
def breadth_first_search(start, goal, transition_graph):
    #TODO: Implement the breadth-first search algorithm. It should return the path as a list of positions
    return None
```

These functions should be implemented by you. Given the start, goal state and the transition graph, it should compute a path using the respective algorithm. Note that the canonical algorithms allow for states to be revisited. However, this may lead to long or infinite loops. You may need to take steps to prevent this.

The transition graph has been mentioned before, but has yet to be defined. It consists of a dictionary, that stores the neighbour cells of each state. It is created by the make_transition_graph function in line 270 of game_engine.py, as per the listing below.

```python
def make_transition_graph(grid, grid_size):
    graph = {}
    for y in range(grid_size[1]):
```

```
        for x in range(grid_size[0]):
            if grid[y][x] != WALL:
                graph[(x, y)] = []
                for direction in get_valid_directions((x, y), grid, grid_size):
                    new_pos = compute_new_pos((x, y), direction)
                    graph[(x, y)].append(new_pos)
    return graph
```

Once a path is computed, the agent must move in it. This is done by the move_in_path function, shown in the listing below.

```
def move_in_path(agent, game_state, path):
    #This function is called by the search agents to move the agent with the first step in the path

    if path is not None and len(path) > 0:
        next_step = path.pop(0) # pop is used to remove the first element of the list and return it

        if next_step[0] > agent['x']:
            actions.go_right(agent, game_state)
        elif next_step[0] < agent['x']:
            actions.go_left(agent, game_state)
        elif next_step[1] > agent['y']:
            actions.go_down(agent, game_state)
        elif next_step[1] < agent['y']:
            actions.go_up(agent, game_state)
```

As previously mentioned, each agent function should implement the original ghost behaviours (with the relaxed constraints) using search methods. Note that most ghosts (all apart from Inky) are chasing a moving target, and thus it will not be feasible to compute a single trajectory and follow it to your destination. Instead, you will need to continuously re-compute your trajectory. Also, be weary of the ghost state changes. Whenever the ghost becomes frightened (or stops being frightened), it should adopt a new strategy and follow a different path. To help you with this, **the goal and path of each ghost should be stored in each ghost object** (see the creation of ghost objects in line 308 of game_engine.py). Also, these variables are reset (i.e., set to None) each time a ghost changes state (see the update_world and ghost_eaten functions of game_engine.py), so that your agent may know when to compute a different path (if it does not do so at each time step).

Complete the code provided, implement and compare the four search algorithms that we covered in class:

- Breadth-first search;
- Depth-first search;

- Greedy search;
- A*

Compare their performance and efficiency. The performance may be assessed through the score and success rate. Note that, in this instance, the lowest the score and success rate, the better. In turn, the efficiency may be measured through the mean path lengths.

Which algorithm is best suited for this application?

3 Submission

This assignment is to be developed by groups of two students. It is due on the 20th of April 2026 and the submission shall be done in InforEstudante. No delayed submissions will be accepted, nor will any other method of submitting.

Regarding the materials to submit, you must deliver the modified code, with your implementation of the search algorithms and ghost agents, as well as a brief report (10 pages max.) containing a detailed description of your implementations. Considering that there is a stochastic component to the behaviour of some agents, your report should also contain a brief analysis of the performance obtained, with the data being collected from a minimum of 30 simulations. You should also include a critical discussion of the algorithms implemented, analysing the performance and efficiency of each one.

4 References

1. **Inteligência Artificial: Fundamentos e Aplicações**
   Ernesto Costa, Anabela Simões

2. **Artificial Intelligence: A Modern Approach**
   Stuart Russel, Peter Norvig